Problema do Papai Noel com pthreads: implementação, experimentos e análise

Tema 11 — Sistemas Operacionais — Prof. Lucca Abbado Neres — FAG
[Integrante 1] · [Integrante 2] · [Integrante 3] — outubro de 2026

1. Problema e regras

Papai Noel dorme até ser acordado por uma de duas demandas: um grupo de **exatamente três elfos** com um pedido ou a chegada das **nove renas**. Implementamos a versão finita: há uma única viagem e cada elfo faz um único pedido. As regras são:

- um atendimento ajuda exatamente 3 elfos, e um elfo novo não pode invadir um grupo já formado ou em atendimento;
- a viagem só é preparada depois que as 9 renas chegam; se renas e grupo estão prontos no momento da decisão, Noel escolhe as **renas**, e uma ajuda já iniciada nunca é interrompida;
- depois da viagem, os grupos restantes são atendidos, e todas as threads terminam quando as duas demandas acabam.
- parâmetros: o elfo *i* (*i* = 0, 1, …) chega em 10 + *i* ms; as renas chegam juntas em 5, 10 ou 30 ms; uma ajuda dura 3 ms e a preparação da viagem dura 5 ms.

2. Solução e modelagem

O programa (C11, `-std=c11 -O2 -Wall -Wextra -pthread`, sem warnings) cria **1 thread Noel, 9 threads rena e N threads elfo**, todas antes de qualquer `pthread_join`. Cada thread se anuncia e espera o **sinal de início** numa variável de condição. Quando todas estão prontas, a *main* grava t0 com `CLOCK_MONOTONIC` e faz broadcast: assim, criar as threads fica fora da medição. Os sleeps (`clock_nanosleep` absoluto, da família nanosleep) servem apenas para simular chegada, ajuda e preparação. Toda a coordenação usa um único mutex e quatro variáveis de condição, sem espera ocupada. Eventos e amostras ficam em vetores na memória e só são exportados para CSV depois dos joins. Os mecanismos são destruídos após os joins.

Elemento	Tipo	Papel	Valor inicial
mutex	pthread_mutex_t	protege todo o estado compartilhado	destravado
cv_largada	pthread_cond_t	sinal de início comum (main ↔ threads)	largada = 0
cv_noel	cond (CLOCK_MONOTONIC)	Noel dorme; timedwait até t0 + timeout	—
cv_elfos	pthread_cond_t	elfo espera seu grupo ficar CONCLUÍDO	—
cv_renas	pthread_cond_t	renas esperam viagem_concluida	—
renas_prontas	int	renas que já chegaram	0
grupos[k]	{membros[3], n, estado}	fila de grupos com id (ABERTO→PRONTO→EM_AJUDA→CONCLUÍDO)	n = 0, ABERTO
grupo_aberto / grupos_fechados / proximo_grupo	int	grupo que aceita elfos / fechados / próximo a atender	0 / 0 / 0
viagem_iniciada, viagem_concluida, abortar	int	estado da viagem; timeout de segurança	0, 0, 0

Formação dos grupos. O elfo entra sempre em `grupos[grupo_aberto]`. O 3º elfo fecha o grupo (PRONTO), avança `grupo_aberto` e sinaliza Noel. Inscrever-se e fechar o grupo acontecem na mesma seção crítica, então o 4º elfo já cai no grupo seguinte. **Decisão de Noel:** com o mutex adquirido, ele testa primeiro *renas_prontas = 9 e a viagem ainda não feita* e só depois a fila de grupos. Também registra cada decisão e conta violações. As invariantes verificadas em tempo de execução são: grupo com ≠ 3 elfos, viagem com < 9 renas, prioridade violada, decisão durante uma ajuda em curso, elfo atendido 0 ou 2+ vezes e invasão de grupo fechado.

```
NOEL
lock; espera largada
loop:
  se viagem_concluida e todos grupos
     concluídos: sai
  renas_ok = (renas==9 e !viagem)
  elfos_ok = (há grupo PRONTO na fila)
  se nenhum: timedwait(cv_noel, limite)
     (ETIMEDOUT → abortar=1; sai)
  se renas_ok: preparar viagem
  senão: ajudar grupos[proximo++]
registra encerramento; broadcast(todos)
```

```
ELFO i
espera largada; dorme até 10+i ms
lock; t_pedido = agora
g = grupo_aberto; inscreve em g
se g.n == 3: g.estado = PRONTO;
  grupo_aberto++; signal(cv_noel)
enquanto g != CONCLUÍDO e !abortar:
   wait(cv_elfos)
unlock
-- Noel: preparar/ajudar = marca
   estado, unlock, sleep 5/3 ms, lock,
   marca fim, broadcast(cv_renas/elfos)
```

```
RENA r
espera largada
dorme até renas_ms
lock; renas_prontas++
se == 9: t_nona = agora;
  signal(cv_noel)
enquanto !viagem_concluida
   e !abortar:
  wait(cv_renas)
  unlock
```

3. Ordem de execução problemática da solução ingênua

(a) Invasão de grupo, num exemplo construído (não executado). Numa solução ingênua com um contador `elfos_esperando`, os elfos 0, 1 e 2 chegam, o contador vai a 3 e Noel recebe o sinal. Antes de Noel adquirir o mutex, o elfo 3 chega e incrementa o contador para 4. Noel acorda, lê 4 e ajuda quatro elfos, ou o elfo 3 entra na ajuda que já começou. A nossa solução evita isso porque o grupo é fechado na mesma seção crítica em que o 3º elfo se inscreve, e esse elfo ainda avança `grupo_aberto`: o elfo 3 nunca enxerga o grupo 1.

(b) Prioridade violada, executada de verdade. A opção `--ingenua` inverte a regra (elfos primeiro) e foi rodada no mesmo cenário do teste funcional (6 elfos, renas em 30 ms, Noel liberado só com as duas demandas prontas). Sequência registrada em `dados/ingenua/eventos.csv`:

t (ms)	evento
12,078	grupo 1 pronto (elfos 0–2)
15,071	grupo 2 pronto (elfos 3–5)
30,106	9ª rena chega: as duas demandas estão prontas
30,113	**decisão: ajudar grupo 1 com as 9 renas prontas → violação**
33,188	**decisão: ajudar grupo 2 → violação**
36,250	só agora a viagem começa a ser preparada

O programa contou **2 violações de prioridade**, status *violacao*, e a resposta às renas foi de 6,145 ms, contra 0,018 ms na solução correta (seção 7). A solução evita esse problema porque testa *renas_ok* antes de *elfos_ok* dentro da mesma seção crítica em que decide.

4. Análise de deadlock e starvation

Não há deadlock. Existe um único mutex, então não há espera circular entre locks. Quem espera numa variável de condição libera o mutex enquanto espera. Cada espera é um laço que reavalia o predicado, o que protege contra acordar espuriamente. Toda mudança que pode tornar um predicado verdadeiro acontece sob o mutex e é seguida de signal/broadcast: o 3º elfo e a 9ª rena acordam Noel, e Noel acorda elfos e renas ao fim de cada atividade. Assim, nenhum aviso se perde. Noel nunca simula trabalho segurando o mutex. Por fim, ao encerrar, Noel faz broadcast em todas as filas. O timeout de segurança (`pthread_cond_timedwait`) marca *abortar* e libera todos, e cada thread testa *abortar* ao acordar.

A espera de um elfo é finita nesta versão. Existe no máximo uma viagem (5 ms). Os grupos são atendidos em ordem FIFO pelo id, e cada ajuda dura 3 ms. Um grupo na posição k da fila espera, no pior caso, $5 + 3(k-1)$ ms, mais o tempo de formação (≤ 2 ms para o primeiro membro). Isso é finito porque N é finito.

Risco de starvation com demandas contínuas. Se as renas voltassem sempre, a prioridade absoluta das renas poderia deixar os grupos de elfos esperando sem limite. Isso aconteceria se as 9 renas estivessem prontas a cada decisão, já que viagens sucessivas sempre venceriam. Possíveis mitigações: *aging* (um grupo que espera mais que um limite ganha prioridade), uma cota (atender ao menos um grupo pendente entre duas viagens) ou um limite de viagens consecutivas. Todas relaxam a regra "renas primeiro" em troca de garantir progresso aos elfos. Entre os próprios elfos não há starvation, porque a fila de grupos é FIFO.

5. Método

Ambiente (`dados/ambiente.txt`): AMD Ryzen AI 5 340 w/ Radeon 840M, 12 CPUs lógicas, 14Gi de RAM, Fedora Linux 43 (Workstation Edition), kernel 7.2.11-100.fc43.x86_64, gcc (GCC) 15.3.1 20260722 (Red Hat 15.3.1-1), (GNU libc) 2.42. Compilação: `cd src && gcc -std=c11 -O2 -Wall -Wextra -pthread *.c -o simulacao`.

Hipóteses (escritas antes das execuções, deduzidas só dos parâmetros). H_A: com as renas chegando junto, a viagem ocupa Noel de 10 a 15 ms, e os grupos ficam prontos a cada 3 ms a partir de 12 ms. Logo T ≈ 15 + 3·(N/3) ms, e a vazão N/T deve **crescer** com N, porque o custo fixo (10 ms até o 1º elfo e 5 ms de viagem) é diluído: previsão de 167, 286 e 444 pedidos/s para 3, 6 e 12 elfos, rumo ao limite de 1000 pedidos/s (3 elfos a cada 3 ms). **H_B:** o p95 da espera deve ser **maior com as renas junto** (≈ 5 ms), porque o grupo 1 fica pronto em 12 ms com Noel ainda preparando a viagem. Antes e depois, Noel está livre quando cada grupo fecha, e a espera se reduz ao tempo de formação do grupo (≈ 2 ms). A resposta às renas deve ser próxima de zero nos três casos, porque Noel está dormindo quando a 9ª rena chega.

Matriz. São 5 configurações distintas: a referência (6 elfos, 10 ms); o conjunto 1 varia os elfos (3, 6, 12) com as renas em 10 ms; o conjunto 2 varia as renas (5, 10, 30 ms) com 6 elfos. Cada uma tem 5 repetições, num total de **25 execuções**, intercaladas por repetição. As sementes 11, 22, 33, 44 e 55 são apenas o id da repetição: o programa não tem aleatoriedade, e a variação vem do escalonador e da latência dos timers. **Piloto:** 15 execuções (3 por configuração), maior T = 35,21 ms. Fixamos o **timeout em 2000 ms** (≈ 56× o maior T). O mecanismo foi verificado com `--timeout-ms 5`: status *timeout*, 0/6 pedidos, encerramento sem travar. Não foi preciso ajustar a escala: as durações (ms) são muito maiores que a latência de acordar uma thread, que é de dezenas de µs (veja a resposta às renas). **Estresse:** 300 execuções extras (3, 12 e 12 elfos com renas em 10, 5 e 30 ms), 300 com status ok.

Métricas. T vai de t0 ao encerramento de Noel. Vazão = pedidos atendidos / (T/1000) s. Espera = do pedido (chegada) ao início da ajuda do grupo. p95 = valor na posição teto(0,95·n) (base 1) da lista ordenada; com n = 6 ou 3 amostras, o p95 coincide com o máximo. Resposta às renas = da 9ª rena ao início da preparação. Utilização de Noel = tempo em ajuda ou preparação / T.

6. Resultados

Taxa de execuções válidas: 25/25 (100%), sem nenhum timeout e com zero violações de prioridade ou de regras em toda a matriz. A tabela resume `dados/tabela_esperas.csv`. As médias são das 5 execuções de cada configuração.

Config.	N	Renas	OK	Vazão média (mín–máx) pedidos/s	Espera média (ms)	p95 médio (mín–máx) ms	Máx (ms)	Resposta renas (µs)	Util. Noel
E03_R10	3	10 ms	5/5	163,4 (160,0–164,3)	4,21	5,21 (5,11–5,56)	5,56	24	44,8%
E06_R10	6	10 ms	5/5	280,9 (280,2–281,3)	4,17	5,20 (5,17–5,25)	5,25	18	52,5%
E12_R10	12	10 ms	5/5	436,1 (435,4–436,9)	4,25	5,37 (5,32–5,41)	5,41	20	63,0%
E06_R05	6	5 ms	5/5	328,7 (328,5–328,9)	1,06	2,10 (2,10–2,11)	2,11	20	61,5%
E06_R30	6	30 ms	5/5	170,0 (168,8–170,6)	1,07	2,11 (2,10–2,12)	2,12	21	32,0%

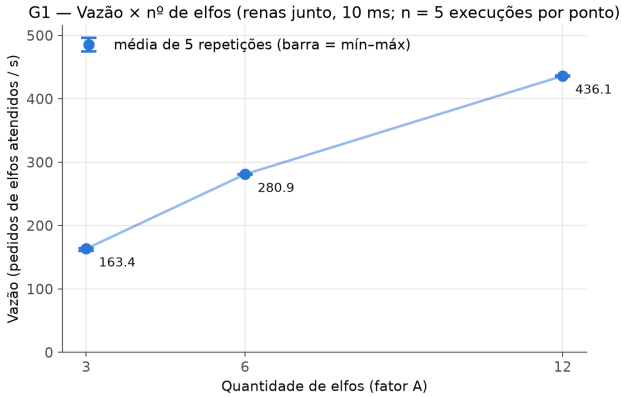


Figura G1. Fixo: renas em 10 ms. Variou: nº de elfos (3, 6, 12), com 5 execuções por ponto (15 no total). A vazão cresce de 163,4 para 280,9 e 436,1 pedidos/s. Isso decorre da fila FIFO de grupos: depois da viagem, Noel atende os grupos em sequência, cada um logo após o anterior, e o custo fixo inicial de 15 ms pesa menos quando N cresce. **Confirma H_A**: o desvio em relação à previsão foi de -2,0%, -1,7% e -1,9%, explicado pela latência de cada sleep. A diferença entre níveis (> 100 pedidos/s) supera muito a variação entre repetições: a maior faixa é a de 3 elfos, 160,0–164,3, por causa de uma execução atrasada (rep 2, T = 18,75 ms).

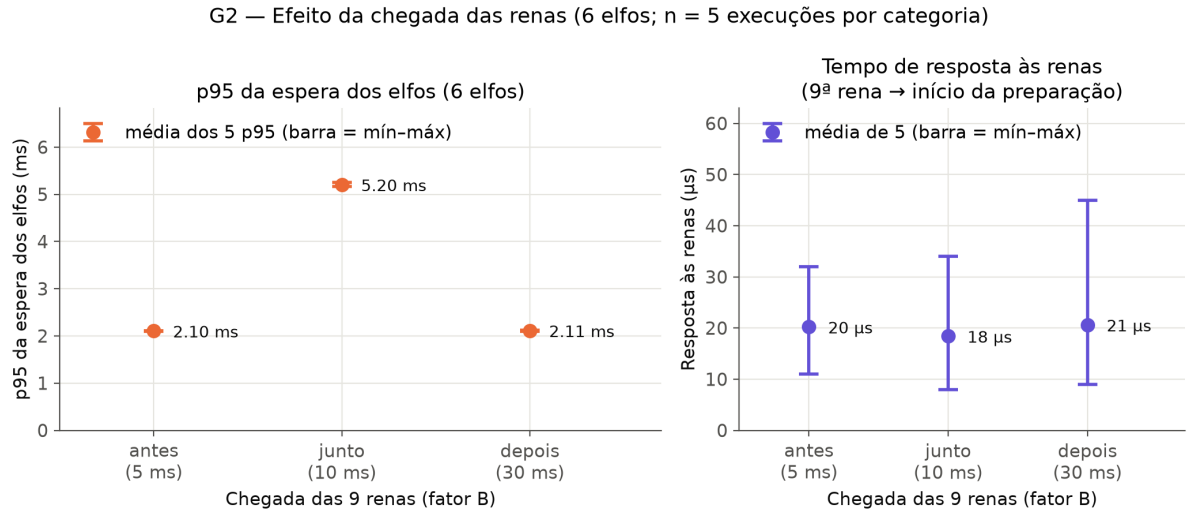


Figura G2. Fixo: 6 elfos. Variou: a chegada das renas (categorias), com 5 execuções por categoria (15 no total). O p95 da espera é 5,20 ms com as renas *junto*, contra 2,10 ms *antes* e 2,11 ms *depois*. Só no caso *junto* a prioridade às renas coloca a viagem (10–15 ms) sobre a formação do grupo 1. Nos outros dois, Noel está livre quando cada grupo fecha, e a espera se resume ao tempo de formação. **Confirma H_B**. A diferença (~3 ms) é muito maior que a faixa das repetições (< 0,1 ms). Painel direito: a resposta às renas fica em dezenas de µs nos três casos (máximo de 45 µs). Esse é o custo de acordar Noel, e as diferenças entre categorias ficam dentro do ruído.

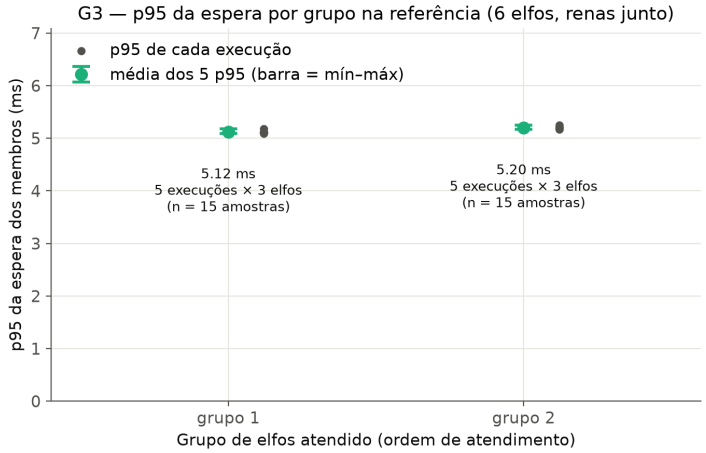


Figura G3. Fixo: a referência (6 elfos, renas junto), com 5 execuções e 3 amostras por grupo em cada uma (n = 15 por grupo). Variou: o grupo atendido. O p95 do grupo 1 é 5,12 ms (5,09–5,18) e o do grupo 2 é 5,20 ms (5,17–5,25). Os dois são praticamente iguais. O grupo 1 espera o fim da viagem, e o grupo 2 espera o fim da ajuda ao grupo 1. Como os grupos chegam a cada 3 ms e cada ajuda também dura 3 ms, o atraso inicial causado pela viagem se propaga sem crescer. A expectativa intuitiva de que grupos posteriores esperariam mais **não se confirmou**. A diferença entre grupos (~0,1 ms, latência acumulada) é da ordem da variação entre repetições.

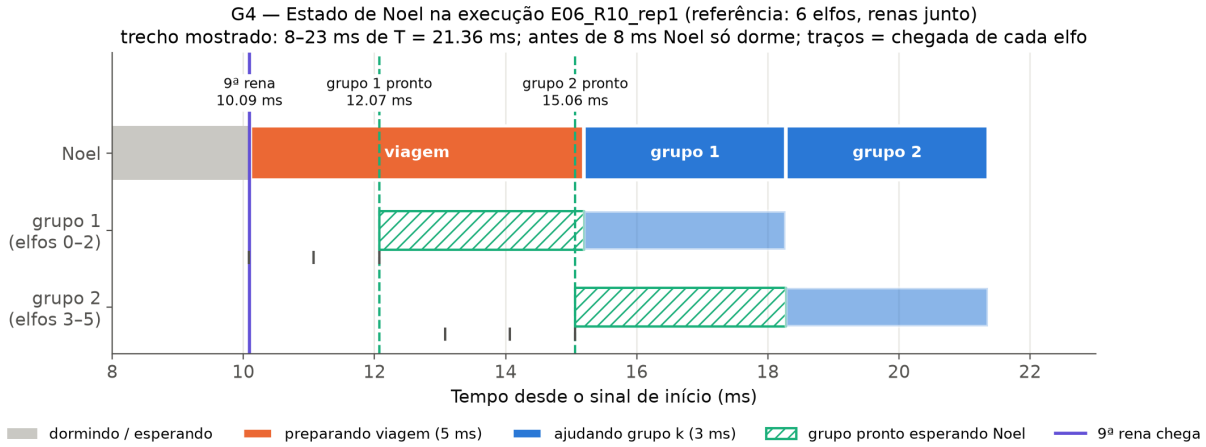


Figura G4. Linha do tempo de uma execução válida da referência (E06_R10_rep1), no trecho de 8 a 23 ms. A 9ª rena chega em 10,09 ms, e Noel prepara a viagem até 15,19 ms. Há disputa nesse trecho: o grupo 1 fica pronto em 12,07 ms e o grupo 2 em 15,06 ms, ambos com Noel ocupado, e esperam (faixa hachurada) sem interromper a preparação. A partir daí Noel fica ocupado sem pausa. A utilização de 52,5% é baixa porque os ~10 ms iniciais não têm nenhuma demanda.

7. Teste funcional de prioridade

No modo `--teste-prioridade`, Noel começa "atrasado": espera em `cv_noel`, sem sleep, até que as 9 renas e pelo menos um grupo estejam prontos. Com 6 elfos e renas em 30 ms, os grupos ficaram prontos em 12,060 e 15,059 ms, e a 9ª rena em 30,455 ms. A primeira decisão, em 30,473 ms, foi a **viagem**, com os grupos atendidos depois. Saída salva em `dados/teste_prioridade/saida.txt`:

```
$ ./simulacao --elfos 6 --renas-ms 30 --rep 1 --semente 11 --out ../dados/teste_prioridade/ --teste-prioridade
teste_E06_R30_rep1: status=ok T=41.927 ms concluidos=6/6 vazao=143.11 pedidos/s espera_p95=25.766 ms
    resposta_renas=0.018 ms viol_prio=0 viol_regras=0 decisoes=3 (com disputa: 1) util_noel=27.1%
[teste-prioridade] disputa na 1a decisão (renas e grupo prontos): SIM
[teste-prioridade] primeira decisão = renas -> OK
[teste-prioridade] elfos atendidos = 6/6 -> OK
[teste-prioridade] violações (prioridade=0, regras=0) -> OK
[teste-prioridade] RESULTADO: PASSOU
codigo_saida=0
```

Resultado: primeira decisão = renas, 6/6 elfos atendidos, 0 violações de prioridade e 0 de regras: **PASSOU**. No mesmo cenário, a política ingênua (seção 3) falha nas duas primeiras decisões.

8. Conclusões e limitações

A solução, com mutex, variáveis de condição e uma fila de grupos com id, cumpriu todas as regras em 25/25 execuções da matriz, em 300 execuções de estresse e no teste funcional, sempre sem travamento. As duas hipóteses se confirmaram. A vazão cresce com N, porque os custos fixos são amortizados. O p95 da espera dos elfos só aumenta quando a viagem coincide com a formação do primeiro grupo: é o preço visível da prioridade às renas, que respondem em dezenas de µs.

- O cenário é dirigido pelo tempo e quase determinístico, por isso a variação entre repetições é pequena: a amplitude (máx − mín) da vazão fica em até 2,7% da média, e o maior caso é E03_R10, com a execução atrasada; nas demais configurações chega no máximo a 1,01%. As sementes só identificam as repetições.
- As medidas incluem a latência dos timers e do escalonador (≈ 0,05–0,5 ms por sleep). Uma execução de 3 elfos teve T = 18,75 ms, contra ~18,3 ms nas outras, por atraso do SO.
- Com 3 ou 6 amostras por execução, o p95 equivale ao máximo. A vazão de N pequeno é penalizada pelos 10 ms iniciais sem demanda, que entram em T por definição.
- Não havia ThreadSanitizer nem Helgrind na máquina (as bibliotecas libtsan/valgrind não estão instaladas). A correção se apoia nas invariantes verificadas em tempo de execução, no estresse e no teste funcional.
- N precisa ser múltiplo de 3 (o programa valida). A versão contínua, com risco de starvation, foi apenas analisada.